

تقرير مشروع مادة البرمجة المتوازية محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

فكرة المشروع

المشروع عبارة عن Backend لنظام تجارة إلكترونية. الهدف منه إدارة المنتجات، المخزون، المحافظ المالية للمستخدمين، وطلبات الشراء، مع التركيز على مفاهيم البرمجة المتوازية والمتطلبات غير الوظيفية مثل سلامة البيانات، إدارة الموارد، المعالجة غير المتزامنة، معالجة الدفعات، وتوزيع الأحمال.

عند تنفيذ عملية شراء، يقوم النظام بالتحقق من رصيد المستخدم والمخزون، ثم يخصم الرصيد وينقص كمية المنتج ويحفظ الطلب في قاعدة البيانات. كل ذلك يتم ضمن معاملة واحدة حتى لا تحدث بيانات غير صحيحة.

معمارية المشروع

تم تقسيم المشروع إلى عدة أجزاء حتى يكون الكود واضحًا ومنظمًا.

قسم product مسؤول عن إدارة المنتجات والمخزون.

قسم wallet مسؤول عن إدارة محافظ المستخدمين والأرصدة.

قسم order مسؤول عن إنشاء طلبات الشراء والتحقق من الرصيد والمخزون.

قسم report مسؤول عن إنشاء تقرير المبيعات ومعالجة البيانات على شكل دفعات.

قسم load مسؤول عن إظهار نسخة التطبيق التي عالجتها الطلب، وذلك لاختبار توزيع الأحمال.

قسم monitoring مسؤول عن مراقبة زمن تنفيذ الدوال باستخدام مفهوم (AOP).

قسم invoicequeue مسؤول عن إدارة طابور مهام الفواتير غير المتزامنة.

التقنيات المستخدمة

الاستخدام	التقنية
Spring Boot	وتشغيل التطبيق REST APIs بناء
Spring Data JPA / Hibernate	التعامل مع قاعدة البيانات وإنشاء الجداول تلقائيًا
MySQL / MariaDB عبر XAMPP	حفظ البيانات بشكل دائم
phpMyAdmin	عرض وإدارة قاعدة البيانات
Postman	اختبار الطلبات
Nginx	توزيع الأحمال بين نسختين من التطبيق
AOP	مراقبة زمن تنفيذ الدوال
Apache JMeter	اختبار الضغط ومحاكاة عدد من المستخدمين المتزامنين
Spring Boot Actuator	مراقبة حالة التطبيق ومقاييس الموارد أثناء التشغيل
Task Manager	مراقبة استخدام المعالج والذاكرة على مستوى الجهاز

بنية المشروع Architecture

تم تقسيم المشروع حسب المسؤوليات حتى يكون الكود واضحًا وسهل التعديل. الشكل العام للتنفيذ هو:

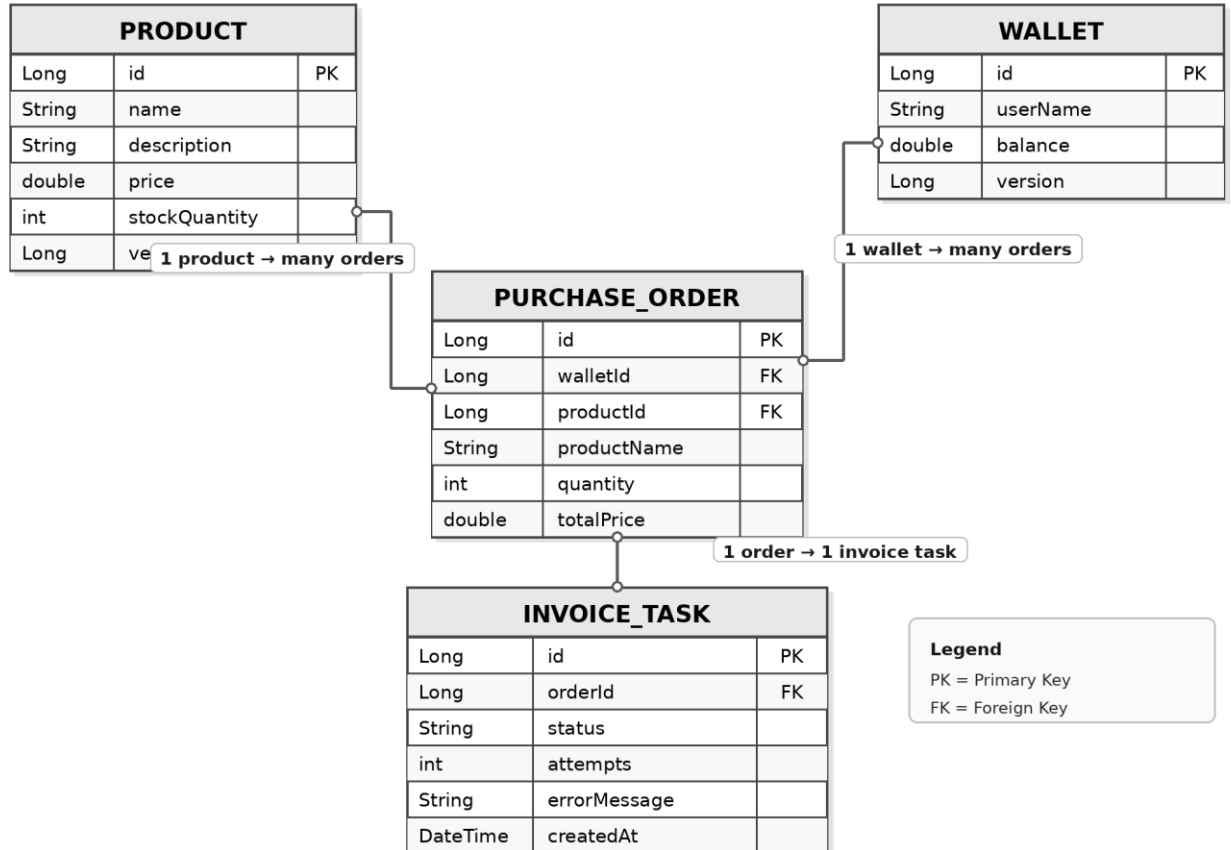
Postman / Client → Controller → Service → Repository → MySQL Database

المسؤولية	المجلد
product	إدارة المنتجات والمخزون
wallet	إدارة محفظة المستخدم ورصيده
order	إنشاء الطلبات وتنفيذ منطق الشراء
report	إصدار تقرير المبيعات ومعالجة البيانات على دفعات
load	معرفة نسخة السيرفر التي عالجت الطلب
monitoring	AOP مراقبة الأداء باستخدام
invoicequeue	إدارة طابور مهام الفواتير غير المتزامنة داخل قاعدة البيانات

مخطط قاعدة البيانات ERD

يوضح المخطط الجداول الأساسية في قاعدة البيانات والعلاقات بينها:

E-Commerce Engine ERD



العلاقة الأساسية هي أن المنتج الواحد يمكن أن يظهر في عدة طلبات، والمحفظة الواحدة يمكن أن تنشئ عدة طلبات شراء. يتم استخدام version في product و wallet لدعم القفل المتفائل.

قاعدة البيانات

تتكون قاعدة البيانات من عدة جداول رئيسية.

جدول product يخزن بيانات المنتجات، مثل اسم المنتج، الوصف، السعر، كمية المخزون، ورقم النسخة المستخدم في القفل المتفائل.

جدول wallet يخزن بيانات محفظة المستخدم، مثل اسم المستخدم، الرصيد، ورقم النسخة المستخدم لحماية الرصيد من التعديل المتزامن.

جدول purchase_order يخزن طلبات الشراء التي يتم إنشاؤها بعد التحقق من الرصيد والمخزون.

جدول invoice_task يخزن مهام إصدار الفواتير غير المترامنة، ويحتوي على حالة المهمة، عدد محاولات التنفيذ، وقت الإنشاء، ووقت المعالجة.

الجدول الأساسية في قاعدة البيانات هي: product و wallet و purchase order و invoice task.

Showing rows 0 - 0 (المجموع 1, استغرق الاستعلام 0.0004 ثانية).

`SELECT \* FROM `wallet`

☐ جانبي
 ☐ تحرير مُضَمَّن
 ☐ [تعديل]
 ☐ [شرح SQL]
 ☐ [أنشئ كود PHP]
 ☐ [تحديث]

25

عدد الأسطر:

☐ عرض الكل

[Extra options](#)

version	user_name	balance	id	
1	Ahmad	400	1	حذف نسخ تعديل

☐ تحقق من الكل

مع المحدد:

☐ تعديل

☐ نسخ

☐ حذف

☐ تصدير

25

عدد الأسطر:

☐ عرض الكل

Showing rows 0 - 1 (المجموع 2, استغرق الاستعلام 0.0002 ثانية).

`SELECT \* FROM `purchase_order`

☐ جانبي
 ☐ تحرير مُضَمَّن
 ☐ [تعديل]
 ☐ [شرح SQL]
 ☐ [أنشئ كود PHP]
 ☐ [تحديث]

25

عدد الأسطر:

☐ عرض الكل

[Extra options](#)

total_price	quantity	product_name	product_id	id	
800	1	Laptop	1	1	حذف نسخ تعديل
1600	2	Laptop	1	2	حذف نسخ تعديل

☐ تحقق من الكل

مع المحدد:

☐ تعديل

☐ نسخ

☐ حذف

☐ تصدير

25

عدد الأسطر:

☐ عرض الكل

Showing rows 0 - 1 (المجموع 2, استغرق الاستعلام 0.0002 ثانية).

`SELECT \* FROM `purchase_order`

☐ جانبي
 ☐ تحرير مُضَمَّن
 ☐ [تعديل]
 ☐ [شرح SQL]
 ☐ [أنشئ كود PHP]
 ☐ [تحديث]

25

عدد الأسطر:

☐ عرض الكل

[Extra options](#)

total_price	quantity	product_name	product_id	id	
800	1	Laptop	1	1	حذف نسخ تعديل
1600	2	Laptop	1	2	حذف نسخ تعديل

☐ تحقق من الكل

مع المحدد:

☐ تعديل

☐ نسخ

☐ حذف

☐ تصدير

25

عدد الأسطر:

☐ عرض الكل

``SELECT * FROM `invoice_task``

☐ جانبي [تحرير مُضمّن] [تعديل] [شرح SQL] [أنشئ كود PHP] [تحديث]

عرض الكل | عدد الأسطر: 25 تصفية الصفوف: ابحث في هذا الجدول :Sort by key لا شيء

Extra options

status	processed_at	order_id	error_message	created_at	attempts	id	
DONE	20:07:36.000000	2026-05-11	4	NULL	20:07:33.000000	2026-05-11	1
DONE	20:10:50.000000	2026-05-11	5	NULL	20:10:45.000000	2026-05-11	1

تحقق من الكل مع المحدد: تعديل حذف نسخ تصدير

الطلبات الأساسية API

الهدف	الرابط	الطلب
POST	/api/products	إضافة منتج جديد
GET	/api/products	عرض جميع المنتجات
GET	/api/products/{id}	عرض منتج حسب الرقم
PUT	/api/products/{id}/stock	تعديل كمية المخزون
POST	/api/wallets	إنشاء محفظة ورصيد للمستخدم
GET	/api/wallets/{id}	عرض محفظة حسب الرقم
POST	/api/orders	إنشاء طلب شراء مع فحص الرصيد والمخزون

منطق إنشاء الطلب

عملية إنشاء الطلب هي أهم عملية في المشروع، لأنها تعدل أكثر من مورد حساس: رصيد المحفظة وكمية المنتج في المخزون. لذلك تم وضعها داخل OrderService وليس داخل Controller.

• جلب المحفظة حسب walletId.

• جلب المنتج حسب productId.

• حساب السعر الكلي بناءً على السعر والكمية.

• التأكد من أن الرصيد كافٍ.

• التأكد من أن المخزون كافٍ.

• خصم المبلغ من المحفظة.

• إنقاص كمية المنتج من المخزون.

• حفظ طلب الشراء.

حماية البيانات من التضارب Concurrent Access & Data Integrity

المخزون والرصيد بيانات مشتركة يمكن تعديلها من أكثر من طلب في نفس الوقت. لذلك تم استخدام Optimistic Locking من خلال @Version داخل Product و Wallet. إذا حصل تعديل متزامن على نفس السجل، يستطيع Hibernate اكتشاف التعارض عن طريق رقم النسخة.

كما تم استخدام createOrderWithRetry لمحاولة إعادة تنفيذ العملية عند حدوث تعارض متزامن.

هذه صورة التنفيذ

```
1 {  
2     "productId": 1,  
3     "productName": "Laptop",  
4     "quantity": 1,  
5     "totalPrice": 800.0,  
6     "id": 3  
7 }
```

سلامة المعاملات Transaction Integrity

لضمان سلامة البيانات، تم استخدام @Transactional داخل OrderService بحيث يتم تنفيذ خطوات عملية الشراء ضمن معاملة واحدة. تتضمن هذه الخطوات خصم رصيد المستخدم، تحديث كمية المخزون، ثم حفظ طلب الشراء. في حال حدوث خطأ أثناء أي خطوة، يتم إلغاء جميع التغييرات السابقة تلقائيًا، وبذلك لا يحدث عدم توافق بين الرصيد والمخزون والطلب.

إدارة الموارد Resource Management

تم تطبيق إدارة الموارد من خلال تحديد عدد خيوط Tomcat وتحديد عدد اتصالات قاعدة البيانات في Hikari Connection Pool. الهدف هو منع التطبيق من استهلاك الموارد بشكل غير محدود عند وجود ضغط أو عدد كبير من الطلبات المتزامنة.

server.tomcat.threads.max=50

server.tomcat.threads.min-spare=10

spring.datasource.hikari.maximum-pool-size=10

المعالجة غير المتزامنة Asynchronous Processing

بعد إنشاء طلب الشراء، لا يتم تنفيذ مهمة إصدار الفاتورة داخل المسار الرئيسي للطلب. يتم تسجيل مهمة في جدول invoice_task، ثم يقوم Worker يعمل في الخلفية بمعالجتها لاحقًا. بهذه الطريقة يحصل المستخدم على الاستجابة بسرعة، وتبقى مهمة الفاتورة قابلة للتتبع من قاعدة البيانات.

تم استخدام مفهوم الطابور غير المتزامن (Asynchronous Queue) داخل قاعدة البيانات، إضافة إلى تشغيل Worker في الخلفية لمعالجة المهام المعقدة.

```
Invoice generated asynchronously for order id: 3  
[AOP PERFORMANCE] AsyncOrderTaskService.generateInvoiceAsync executed in 2016 ms  
[AOP PERFORMANCE] OrderService.createOrderWithRetry executed in 2102 ms  
[AOP PERFORMANCE] OrderController.createOrder executed in 2102 ms
```

الصورة: تنفيذ مهمة إصدار الفاتورة في الخلفية بعد إنشاء طلب الشراء، مما يثبت تطبيق المعالجة غير المتزامنة دون تعطيل استجابة المستخدم.

```

        it1_0.status
    from
        invoice_task it1_0
    where
        it1_0.status=?
    order by
        it1_0.created_at
    limit
        ?
    Processing invoice task for order id: 5
    Invoice task completed for order id: 5
    Hibernate:
        update
            invoice_task

```

الشكل: معالجة مهمة إصدار الفاتورة من خلال Asynchronous Queue، حيث قام الـ Worker بقراءة المهمة الخاصة بالطلب رقم ٥ من جدول invoice_task ثم نفذها في الخلفية.

25 | عرض الكل | عدد الأسطر: | Extra options

us	processed_at	order_id	error_message	created_at	attempts	id	
NE	20:07:36.000000	2026-05-11 4	NULL	20:07:33.000000	2026-05-11 1	1	حذف
NE	20:10:50.000000	2026-05-11 5	NULL	20:10:45.000000	2026-05-11 1	2	حذف

تصدير | حذف | نسخ | تعديل | مع المحدد: | تحقق من الكل

معالجة الدفعات Batch Processing

تم إنشاء endpoint لتقرير المبيعات يقوم بقراءة الطلبات ومعالجتها على دفعات صغيرة Chunks بدل معالجة كل البيانات دفعة واحدة.

GET /api/reports/daily-sales

```

    purchase_order po1_0
    Processed chunk from index 0 with size 2
    Processed chunk from index 2 with size 1

```

الصورة معالجة طلبات الشراء على شكل دفعات Chunks أثناء تنفيذ تقرير المبيعات، بدل معالجة كل السجلات دفعة واحدة.

توزيع الأحمال Load Distribution

تم استخدام Nginx كـ Load Balancer لمحاكاة توزيع الطلبات على أكثر من خادم. تم تشغيل نسختين من تطبيق Spring Boot؛ النسخة الأولى على المنفذ ٨٠٨٢ باسم server-8082، والنسخة الثانية على المنفذ ٨٠٨٣ باسم server-8083. بعد ذلك تم ضبط Nginx ليستقبل الطلبات على المنفذ ٨٠٩٠، ثم يوزعها بين النسختين باستخدام استراتيجية Round Robin، أي بالتناوب بين الخوادم.

تم إثبات توزيع الأحمال من خلال تنفيذ الطلب التالي عدة مرات:

GET http://localhost:8090/api/load/instance

عند تكرار تنفيذ الطلب، تظهر الاستجابة أحياناً من server-8082 وأحياناً من server-8083، وهذا يثبت أن Nginx يوزع الطلبات بين النسختين.

```

"instanceName": "server-8083",
"serverPort": "8083",
"message": "Request handled by server-8083",
"timestamp": "2026-05-11T19:05:17.533408600"

```

مراقبة الأداء باستخدام AOP

تم استخدام AOP لمراقبة زمن تنفيذ الدوال المهمة بدون وضع كود قياس الوقت داخل كل دالة. الملف المسؤول هو .PerformanceMonitoringAspect.java

مثال من الـ Terminal:

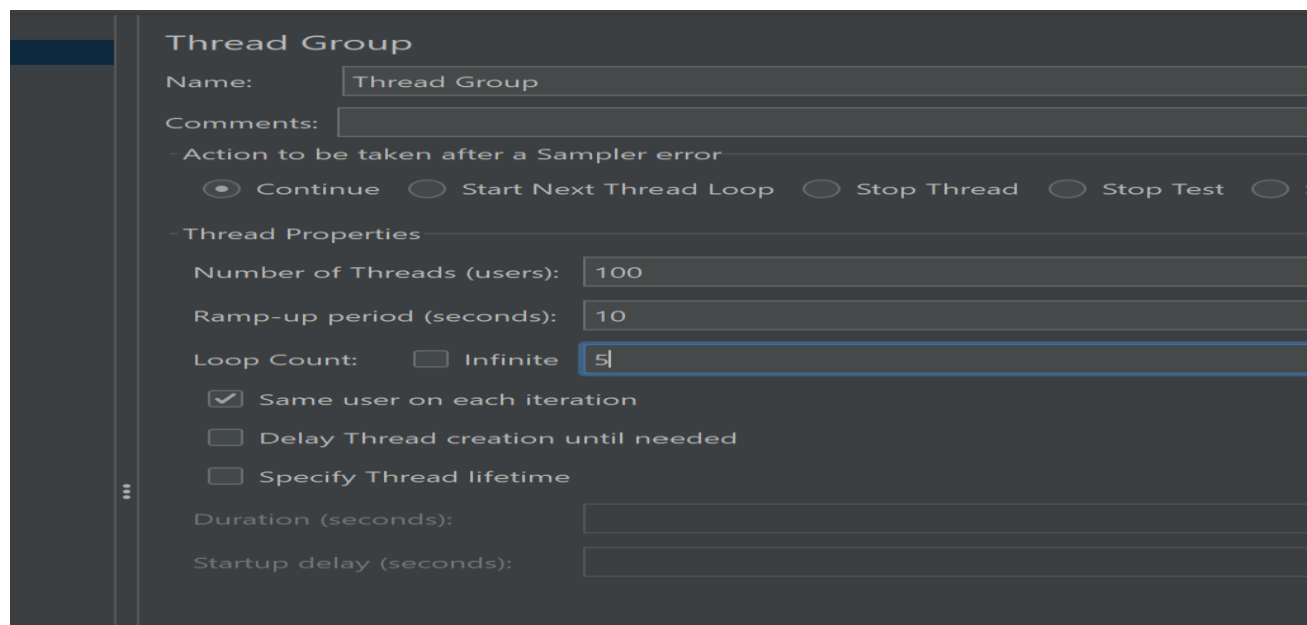
```
[AOP PERFORMANCE] WalletController.getWalletById executed in 75 ms
```

طريقة التشغيل

- تشغيل برنامج XAMPP، ثم تشغيل خدمة قاعدة البيانات MySQL.
- تشغيل النسخة الأولى من التطبيق على المنفذ ٨٠٨٢ باسم server-8082.
- تشغيل النسخة الثانية من التطبيق على المنفذ ٨٠٨٣ باسم server-8083.
- تشغيل Nginx ليعمل كموزع أحمال بين النسختين.
- إرسال الطلبات من برنامج Postman إلى الرابط التالي:

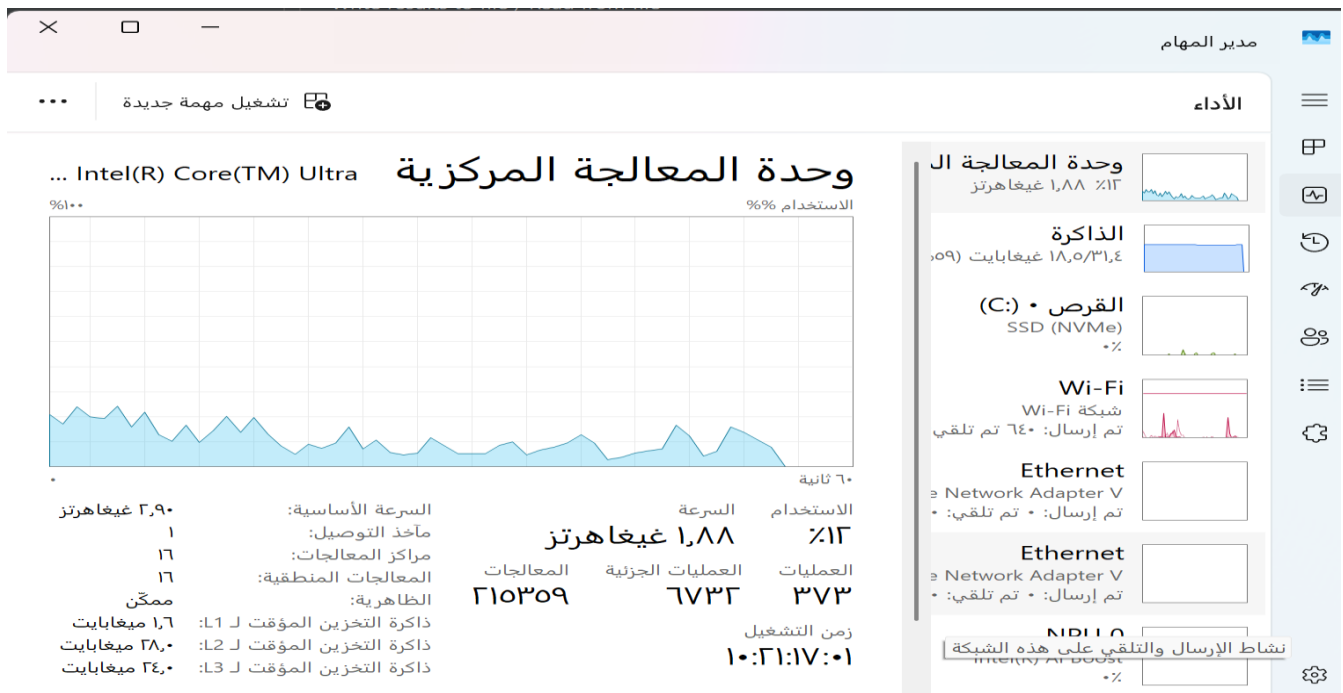
<http://localhost:8090>

بعض الصور لاثبات أداء البرنامج:



Summary Report										
Name:	Summary Report									
Comments:										
Write results to file / Read from file										
Filename	C:\Users\Zenbook\Desktop\ecommerce-engine\ecommerce_test_results\jtl							Browse...	Log/Display Only:	☐ Errors ☐ Successes Configure
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get Products	500	6	1	34	3.34	0.00	0.2/sec	19.1	6.32	388.0
Get Wallets	500	5	2	49	3.93	0.00	0.2/sec	22.0	6.29	408.0
Load Instance	500	1	0	27	1.37	0.00	0.2/sec	14.24	6.09	289.6
Actuator Health	500	4	1	13	1.50	0.00	0.2/sec	34.01	6.49	702.0
TOTAL	2000	4	0	49	3.39	0.00	2.0/sec	89.90	20.10	409.4

الشكل في الأعلى: الشكل في الأعلى: نتائج اختبار الضغط باستخدام Apache JMeter، حيث تم إعداد Thread Group لمحاكاة ١٠٠ مستخدم، مع Loop Count = 5 لكل مستخدم. تم اختبار عدة endpoints مثل عرض المنتجات، عرض المحافظ، فحص حالة التطبيق، واختبار توزيع الأحمال عبر Nginx. تظهر النتائج أن نسبة الأخطاء ٠,٠٠٪ لجميع الطلبات، مع تنفيذ ٢٠٠٠ طلب بالمجموع، مما يدل على قدرة النظام على التعامل مع عدد كبير من الطلبات المتزامنة دون فشل.



الشكل في الأعلى: مراقبة استهلاك المعالج أثناء تنفيذ اختبار الضغط باستخدام JMeter، حيث تم إرسال عدد كبير من الطلبات إلى النظام ومتابعة تأثيرها على موارد الجهاز.

الشكل: عرض المقاييس المتاحة في Spring Boot Actuator، حيث تظهر مقاييس خاصة بالتطبيق والقرص وال-Executor، والتي تساعد على مراقبة الموارد والمهام غير المتزامنة أثناء التشغيل.

The screenshot shows a web browser window with the address bar displaying `http://localhost:8082/actuator/metrics`. The browser's developer tools are open, showing the `GET` request to the same URL. The response is a `200 OK` with a status of `200 OK`, a response time of `26 ms`, and a content size of `1.74 KB`. The response body is displayed in JSON format, showing a list of metric names.

Query Params

Key	Value	Description
Key	Value	Description

Body

```
{
  "names": [
    "application.ready.time",
    "application.started.time",
    "disk.free",
    "disk.total",
    "executor.active",
    "executor.completed",
    "executor.pool.core",
    "executor.pool.max",
    "executor.pool.size",
    "executor.queue.remaining",
    "executor.queued",
  ]
}
```

الشكل: مراقبة عدد الخيوط الحية داخل تطبيق Spring Boot باستخدام Actuator، حيث يظهر أن عدد الخيوط الحالية هو ٢٥ خيطاً أثناء تشغيل التطبيق.

```
{
  "availableTags": [],
  "baseUnit": "threads",
  "description": "The current number of live threads including both
    daemon and non-daemon threads",
  "measurements": [
    {
      "statistic": "VALUE",
      "value": 25.0
    }
  ],
  "name": "jvm.threads.live"
```

توضح صور Postman تنفيذ الطلبات الأساسية، وتوضح صور phpMyAdmin حفظ البيانات في قاعدة MySQL، وتوضح صور Terminal عمل الـ Worker و AOP، وتوضح صور JMeter نتائج اختبار الضغط، بينما توضح صور Actuator مراقبة الموارد من داخل Spring Boot.

تم توثيق التنفيذ من خلال صور من Postman و phpMyAdmin و Terminal و JMeter و Actuator.

مراقبة الموارد باستخدام Spring Boot Actuator

لمراقبة موارد التطبيق من داخل إطار العمل نفسه، تم استخدام أداة Spring Boot Actuator. توفر هذه الأداة روابط جاهزة لعرض حالة التطبيق والمقاييس الخاصة به أثناء التشغيل.

تم استخدام الرابط `actuator/health/` للتأكد من أن التطبيق وقاعدة البيانات يعملان بشكل صحيح، حيث ظهرت الحالة UP.

كما تم استخدام الرابط `actuator/metrics/` لعرض قائمة المقاييس المتاحة، مثل استهلاك ذاكرة JVM، عدد الخيوط الحية، وحالة Executor المستخدم في المهام غير المتزامنة.

يساعد Actuator على مراقبة موارد التطبيق من داخل Spring Boot، بدل الاعتماد فقط على أدوات نظام التشغيل.

اختبار الضغط باستخدام Apache JMeter

تم استخدام أداة Apache JMeter لمحاكاة ١٠٠ مستخدم متزامن يرسلون طلبات إلى النظام عبر Nginx.

تم ضبط الاختبار مع تكرار الطلبات، وذلك لقياس قدرة النظام على التعامل مع الضغط. من خلال تقرير Summary Report في JMeter تم عرض عدد الطلبات، متوسط زمن الاستجابة، نسبة الأخطاء، ومعدل تنفيذ الطلبات في الثانية.

أثناء الاختبار، تمت مراقبة موارد التطبيق باستخدام Spring Boot Actuator، مثل عدد الخيوط الحية واستهلاك الذاكرة وحالة Executor. كما تم استخدام Task Manager لمراقبة استهلاك المعالج والذاكرة على مستوى الجهاز.

طابور مهام الفواتير غير المتزامنة

تم تطوير المعالجة غير المتزامنة في المشروع لتصبح أقرب إلى مفهوم Asynchronous Queue. بدل تنفيذ مهمة إصدار الفاتورة مباشرة داخل مسار طلب الشراء، يتم تسجيل مهمة جديدة داخل جدول invoice_task بحالة PENDING.

بعد ذلك يقوم Worker يعمل في الخلفية بشكل دوري بقراءة المهام التي حالتها PENDING، ثم يعالجها ويغير حالتها إلى DONE. بهذه الطريقة لا ينتظر المستخدم تنفيذ مهمة إصدار الفاتورة، كما يمكن تتبع حالة المهمة من قاعدة البيانات.

هذا الحل مناسب للمشروع الحالي لأنه مشروع واحد متكامل Monolithic، ولا يحتاج إلى Message Broker خارجي مثل RabbitMQ أو Kafka، لكنه يوضح فكرة الطابور غير المتزامن بشكل مبسط داخل قاعدة البيانات.

الفرق بين Asynchronous و Asynchronous Queue

المقصود بـ Asynchronous هو تنفيذ المهمة في الخلفية دون انتظارها داخل الطلب الرئيسي، مثل استخدام التعليمة Async@.

أما Asynchronous Queue فهي تضيف طبقة طابور تحفظ المهام قبل تنفيذها، بحيث يمكن تتبع حالتها ومعالجتها لاحقاً.

في المشروع الحالي تم استخدام جدول invoice_task كطابور بسيط. يتم تسجيل مهمة الفاتورة في الجدول بحالة PENDING، ثم يقوم Worker بمعالجتها لاحقاً وتغيير حالتها إلى DONE. هذا يجعل المهمة قابلة للتتبع من خلال حالات واضحة مثل PENDING و PROCESSING و DONE و FAILED.

ملخص التحقق العملي

تم توثيق التنفيذ من خلال صور من Postman و phpMyAdmin و Terminal و JMeter و Actuator.

توضح صور Postman تنفيذ الطلبات الأساسية، وتوضح صور phpMyAdmin حفظ البيانات في قاعدة MySQL، وتوضح صور Terminal عمل Worker و AOP، وتوضح صور JMeter نتائج اختبار الضغط، بينما توضح صور Actuator مراقبة الموارد من داخل Spring Boot.